

1팀\_불량난 준석이

# 생산라인 모니터링 채팅 프로그램

실시간 불량 검출과 자동 제어를 통한 스마트 생산라인 구현

팀장:이준석 | 부팀장:김수영 | 팀원:김동호, 이승환, 장효준, 최지윤

# INDEX

## 목차



01

팀원 구성 및 역할

03

프로젝트 개요 및 목적

05

프로젝트 일정

07

순서도

09

시연 영상

11

느낀점

02

프로젝트 선정 이유

04

개발 환경 및 사용 언어

06

구성도

08

기능구현

10

문제점 및 개선 사항

12

Q&A

# 01 팀원 구성 및 역할

## 팀장 / 이준석

- UI 제작
- 시스템 설계
- 순서도 제작

## 부팀장 / 김수영

- 현장 관리자 기능 설계 및 제작
- MFC 통신 모듈 설계
- RS-232 통신 모듈 설계
- 비상 정지 시스템 설계 및 제작
- 전체적인 통신 시스템 설계
- 현장 관리자 GUI 제작
- SQLite 연동 및 설계

## 팀원 / 김동호

- YOLO 데이터 라벨링 및 학습
- 불량 분류 하드웨어 제작
- HSV 채널 변경 및 불량 분류

## 팀원 / 이승환

- 전체 회로 설계 및 제작
- 라즈베리파이를 이용한 모터 및 센서 제어

## 팀원 / 장효준

- C# 윈도우폼 기능 설계 및 제작
- YOLOv8 사용하여 불량 분류
- HSV 채널 변경 및 불량 분류

## 팀원 / 최지윤

- Flask 기능 설계 및 제작
- Flask 통신 모듈 설계
- 채팅 시스템 설계 및 제작
- 비상 정지 시스템 설계 및 제작
- 관리자 GUI 제작
- RS-232 통신 모듈 설계
- MYSQL 연동 및 설계

## 02 프로젝트 선정 이유

### 1. 수작업 검사의 한계

섬유 원단의 품질검사는 숙련된 작업자가 육안으로 결함을 찾아내는 방식에 의존해 왔으나, 수작업 검사의 정확도는 평균 60~70%에 불과하며 미세한 결함이나 직조 불균형, 코팅 불량 등을 감지하는데 한계가 있다. 고속 생산 환경에서는 피로 누적으로 인한 집중력 저하로 오차 발생 가능성이 더욱 높아진다.

출처 : Textile Research Journal / 비전시스템 (visionsystem.kr)

### 3. AI 기반 품질관리로의 전환

AI 기반 머신비전 시스템이 수작업 대비 추가로 20~30%의 결함을 더 감지할 수 있는 것으로 나타났다. 정부는 2030년까지 섬유패션 산업의 디지털 전환 수준을 60%(현재 35%)까지 높이겠다는 목표를 제시하였다.

출처 : Textile Research Journal / 산업통상자원부 (2024.8)

### 2. 숙련 노동자 감소 및 고령화

저출산·고령화에 따른 현장의 숙련노동자가 감소하고 있으며, 이에 따라 기존 수작업 품질검사 체계의 유지가 점점 어려워지고 있다. 고령화로 인한 생산성 저하 문제를 해결하기 위해 AI와 자동화 기술 도입이 필수적인 상황이다.

출처 : 과학기술정보통신부·중소벤처기업부 (2025.11) / goover.ai 섬유·의류 산업 보고서

### 4. YOLO 기반 원단 불량 검출 연구 활성화

자동화된 원단 불량 검출은 품질 관리 향상, 수작업 감소, 섬유 산업의 효율 최적화에 핵심적이며, YOLO 모델을 활용한 실시간 원단 결함 검출 연구가 활발히 진행되고 있다.

출처 : Mao & Hong, "YOLO Object Detection for Real-Time Fabric Defect Inspection in the Textile Industry", Sensors, 2025 (DOI: 10.3390/s25072270)

## 03 프로젝트 개요 및 목적

### ⚙ 확장성 및 유지보수

- Flask 서버를 중심으로 Web, MFC, C# WinForm 등 각 기능을 독립된 모듈로 분리하여 설계
- C# WinForm을 별도 구성하여 현장 카메라 모듈의 독립적인 수정 및 교체 가능
- 특정 기능 변경 시 전체 시스템에 영향을 주지 않는 구조
- 향후 검출 항목 추가나 장비 확장에도 유연한 대응 가능

### 🗄 데이터베이스 활용

- 전체 관리자 정보, 불량 검출 이력, 채팅 데이터 관리를 위한 데이터베이스 구조 설계
- Flask 서버 연동을 통한 불량 발생 기록의 저장 및 조회 구성
- 관리자 간 채팅 내역 관리를 통한 원활한 소통 환경 확보

### 🤖 AI 불량 검출

- YOLOv8 모델을 활용한 제품 불량 자동 검출 기능 구현
- 현장 카메라를 통한 실시간 영상 수집 및 서버 전송 구성
- 수작업 검수 대체를 통한 검출 속도 및 정확도 향상

### 🖱 하드웨어 연동

- 라즈베리파이를 활용한 현장 장비 제어 및 센서 데이터 수집 기능 구현
- MFC 기반 현장관리자 프로그램과 라즈베리파이 간 실시간 통신 구성
- 원격 모니터링 및 현장 제어가 가능한 통합 환경 구현

## 04 개발환경 및 사용언어

### ✓개발 환경 및 언어



#### 환경

- Visual Studio Code
- Visual Studio

#### 언어

- C#, C++, Python, HTML/CSS/JS

#### 제작

- WinForm, MFC, Flask

### ✓통신 프로토콜



- TCP/IP
- Socket.IO (WebSocket 기반)
- HTTP
- RS-232

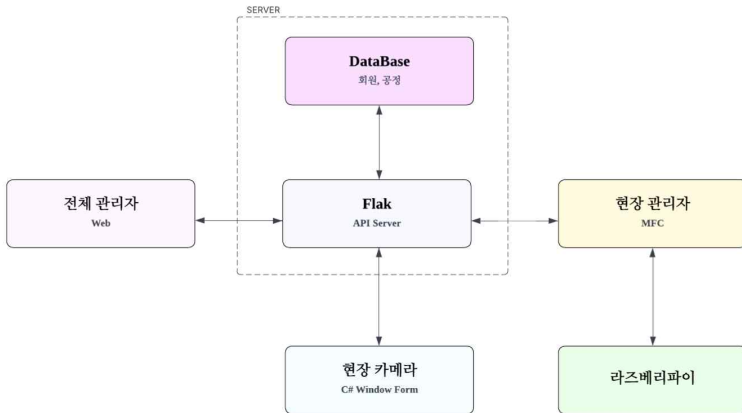
### ✓플랫폼 및 데이터베이스



- Raspberry Pi
- MySQL
- SQLite

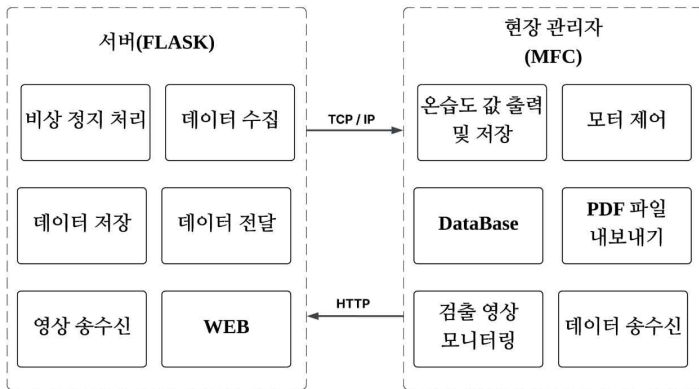
[illegible]

## 06 구성도

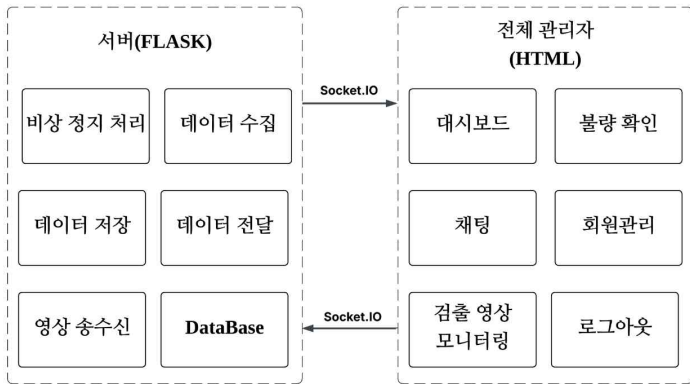




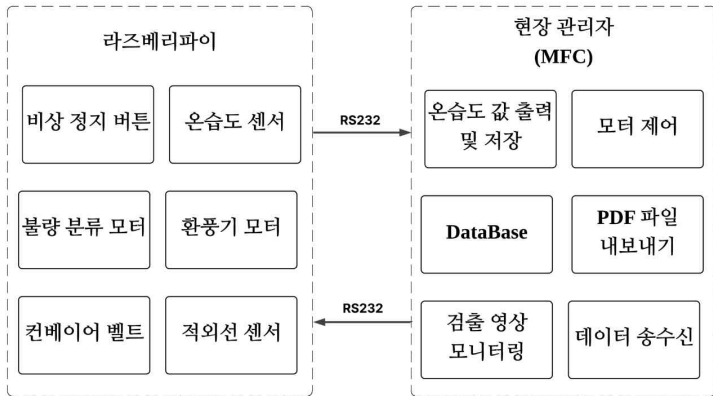
## 06 구성도



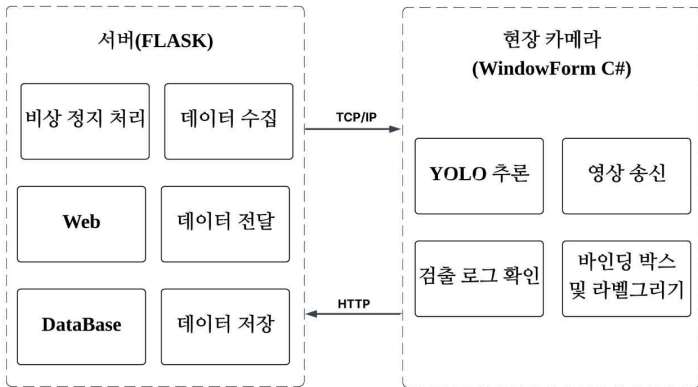
## 06 구성도



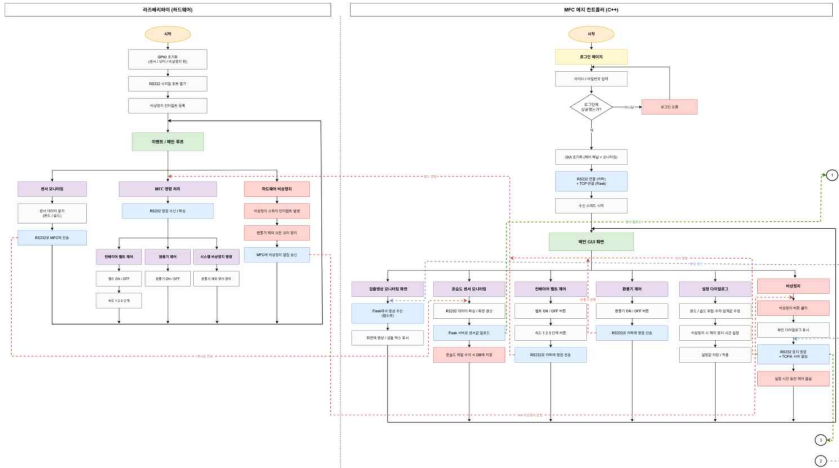
## 06 구성도



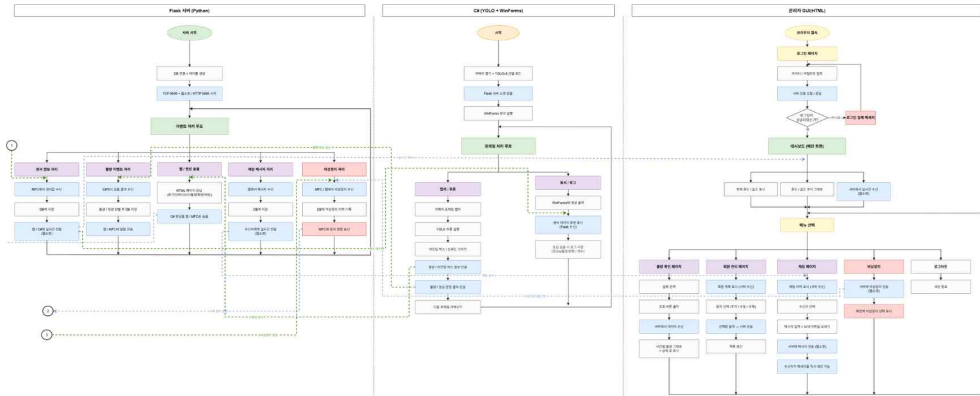
## 06 구성도



## 07 순서도



# 07 순서도



## 08 기능 구현 C# Windowform - 현장 카메라

**FABRIC DEFECT DETECTOR**

모니터링 (컨베이어)

검출 상태 : DEFECT

검출 로그

DEFECT

MODE: WAIT SAMPLE OUT 0/3 (bg=true hsv\_arco=0)

thread 0.94 hole 0.96 thread 0.87

Defect Rate: 100.0% (1/1)

| 시간       | 종류           | 갯수 |
|----------|--------------|----|
| 17:39:02 | hole, thread | 3개 |

1. 중앙 ROI: ROI안에 옷감이 들어올 시 검사 시작
2. 검출 상태 : 정상/검출 상태에 따라 OK, DEFECT 텍스트가 출력
3. 결함일 때 : DEFECT / 정상일 때 : OK
4. MODE : 각 상태에 따라 다른 텍스트 출력
5. DEFECT Rate: 현재 불량률
6. 연결 상태 : 전체관리자와 연결 상태에 따라 버튼의 색깔 및 텍스트 변경
7. DataGridView : 정상/결함이 검출될 때마다 발견된 시간, 결함 종류, 결함 갯수 표기

## 08 기능 구현 C# Windowform - 현장 카메라

Flask 서버에 연결을 위해 필요한 HttpClient

```
private static readonly HttpClient httpClient = new HttpClient()
{
    Timeout = TimeSpan.FromMilliseconds(500)
};
```

결함 탐지용 ONNX 모델 불러오기

```
private void Form1_Load(object sender, EventArgs e)
{
    try
    {
        string modelPath = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, ONNX_MODEL);
        if (!File.Exists(modelPath))
        {
            MessageBox.Show(
                $"ONNX 모델 파일이 없습니다! 경로: {modelPath}\n\n" +
                "1) python convert_to_onnx.py 실행\n" +
                "2) best.onnx를 bin/Debug에 복사",
                "모델 없음", MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }
        detector = new DefectDetector(modelPath);
        this.Text = "Fabric Defect Detector - 모델 로드 완료";
    }
}
```

현재 영상을 HTTP를 이용해 Flask 모니터링 화면에 비동기 전송

```
private async Task SendLiveStreamOptimized(Mat mat, bool isDefect, string mode)
{
    try
    {
        byte[] jpg = mat.ToBytes(".jpg", new int[] { (int)ImwriteFlags.JpegQuality, 70 });
        string b64 = Convert.ToBase64String(jpg);

        var payload = new
        {
            image_data = b64,
            is_defective = isDefect,
            status = mode,
            timestamp = DateTime.Now.ToString("HH:mm:ss.fff")
        };

        string json = System.Text.Json.JsonSerializer.Serialize(payload);
        var content = new StringContent(json, Encoding.UTF8, "application/json");

        var response = await httpClient.PostAsync(MONITOR_URL, content, _cts.Token);
        if (response.IsSuccessStatusCode)
        {
            if (lblConnStatus.Text != "CONNECTED") UpdateConnectionUI(true, "CONNECTED");
            response.Dispose();
        }
    }
}
```



## 08 기능 구현 C# Windowform - 현장 카메라

불량 검출 관련 코드 YOLO을 우선적으로 실행후 HSV를 실행

```
private (List<DetectBox> boxes, string mode) DetectPriority(Mat roi, int ox, int oy)
{
    var yoloBoxes = RunYolo(roi, ox, oy);
    if (yoloBoxes.Count > 0)
    {
        _fallbackCounter = 0;
        return (yoloBoxes, "YOLO");
    }

    _fallbackCounter++;
    if (_fallbackCounter % FallbackEveryN != 0)
        return (new List<DetectBox>(), "YOLO");

    var hsvBoxes = DetectStainHsv(roi, ox, oy);
    return (hsvBoxes, "HSV");
}
```

추론을 통해 신뢰도가 가장 높은 값을 검출 종류로 지정

```
var inputs = new List<NamedOnnxValue> { NamedOnnxValue.CreateFromTensor(inputName, tensor) };
using var results = session.Run(inputs);
var output = results.First().AsTensor<float>();
var dims = output.Dimensions.ToArray();

float maxScore = 0; int maxCls = 0;
for (int c = 0; c < nc; c++)
{
    float s = output[0, 4 + c, i];
    if (s > maxScore) { maxScore = s; maxCls = c; }
}

if (maxScore < ConfThreshold) continue;

string clsName = maxCls < ClassNames.Length ? ClassNames[maxCls] : maxCls.ToString();
if (!UseYoloStain && clsName == "stain") continue;
if (clsName != "hole" && clsName != "thread" && clsName != "stain") continue;
```

## 08 기능 구현 C# Windowform - 현장 카메라

### 결함 검출 시 결함 정보를 전송

```
if (finalIsDefect && !isSampleLocked)
{
    isSampleLocked = true;
    lockedFabricInfo = fabricInfo;
    totalDefect++;
    totalInspected++;

    Mat defectCapture = mat.Clone();

    _ = SendDefectToFlask(defectList);

    string defectTypes = string.Join(" ", defectList.Select(d => d.name).Distinct());
    int defectCount = defectList.Count;

    SetStatus(true, false);
    AddLogToGrid(defectTypes, defectCount.ToString() + "개", true);
    //AppendLog(defectTypes, defectCount);

    Debug.WriteLine("🐛 불량 샘플 확정");
}
```

### Flask 서버로 결함 정보를 순차적으로 전송

```
private async Task SendDefectToFlask(List<(string name, float conf)> defectList)
{
    try
    {
        string defectTypes = string.Join(" ", defectList.Select(d => d.name).Distinct());
        int defectCount = defectList.Count;

        var payload = new
        {
            timestamp = DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss"),
            defect_type = defectTypes,
            defect_count = defectCount
        };

        string json = System.Text.Json.JsonSerializer.Serialize(payload);
        var content = new StringContent(json, Encoding.UTF8, "application/json");

        string flaskUrl = MONITOR_URL.Replace("http://192.168.0.30:5000/defect", "http://192.168.0.30:5000/api/defect");
        await httpClient.PostAsync(flaskUrl, content);

        Debug.WriteLine($"[Flask DB] 불량 전송 완료 - 유형: {defectTypes}, 개수: {defectCount}");
    }
    catch (Exception ex)
    {
        Debug.WriteLine($"[Flask DB] 전송 실패: {ex.Message}");
    }
}
```

## 08 기능 구현 C# Windowform - YOLO\_학습(라벨링)

Uploaded on 04/22/26 at 1:55 pm

Labeler: 김동만

222 Images

Uploaded on 04/22/26 at 11:00 am

Labeler: 김동만

227 Images

Uploaded on 04/20/26 at 3:35 pm: Job 2

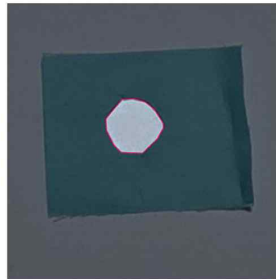
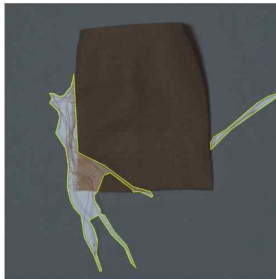
Labeler: 김동만

31 Images

Uploaded on 04/20/26 at 3:35 pm

Labeler: 김동만

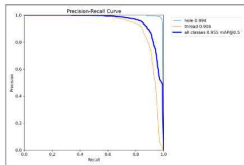
47 Images



라벨링 데이터

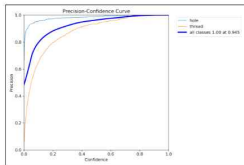
불량품 라벨링

## 08 기능 구현 C# Windowform - YOLO\_학습



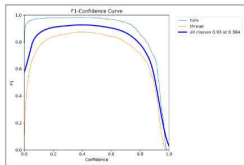
Box PR 그래프

모델의 정확도(유류 검출 및 판별률)를 보여준다.



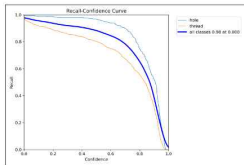
Box P 그래프

검출한 옷감 중에서 실제로 불량인 비율을 보여준다.



Box F1 그래프

PR 모델의 정밀함(P)과 리콜값(R)의 평균을 나타낸다.



Box R 그래프

실제로 있는 불량을 얼마나 놓치지 않고 검출했는지 확인한다.

- 구멍
- 실밥
- 두 값의 평균

왜 실밥이 구멍보다 더 아래에 떨어져 있는가?

구멍은 실밥과 달리 단순한 객체

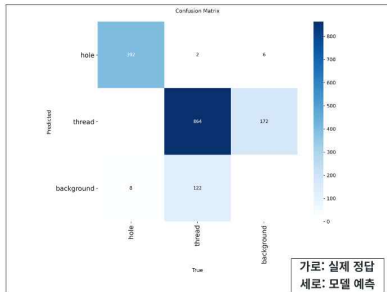
구멍: 모양이 단순하고 균일, 형태가 뚜렷함

실밥: 구조가 복잡하고 얇고 깊. 또한 조명에 따라 가시성이 변함

→ 실밥 검출이 어려워 구멍보다 정확도가 낮다.

## 08 기능 구현 C# Windowform - YOLO\_학습

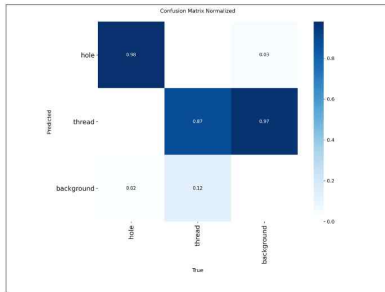
### Confusion Matrix



모델의 검출률을 보여주는 표

구멍: 392개 정답 | 2개 오인 | 8개 인식 실패  
실밥: 864개 정답 | 2개 오인 | 122개 인식 실패  
배경: 172개 오인

### Normalized



Confusion Matrix의 비율을 보여주는 표

구멍 인식률: 98%  
실밥 인식률: 87%  
배경: 97% 실밥으로 오인

## 08 기능 구현 C# Windowform - YOLO\_학습

|           |  |  |  |
|-----------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| 속도        |                                                                                   |                                                                                    |                                                                                     |
| 이미지 처리 시간 | 1.06ms                                                                            | 0.99ms                                                                             | 1.05ms                                                                              |
| 정확도       |                                                                                   |                                                                                    |                                                                                     |
| mAP50     | 34.3%                                                                             | 37.3%                                                                              | 39.5%                                                                               |
| 지원되는 작업   |                                                                                   |                                                                                    |                                                                                     |
| 객체 감지     | ✓                                                                                 | ✓                                                                                  | ✓                                                                                   |
| 이미지 분류    | ✗                                                                                 | ✓                                                                                  | ✓                                                                                   |
| 인스턴스 분할   | ✗                                                                                 | ✓                                                                                  | ✓                                                                                   |
| 객체 추적     | ✗                                                                                 | ✓                                                                                  | ✓                                                                                   |
| 자세 추정     | ✗                                                                                 | ✓                                                                                  | ✓                                                                                   |
| OBB 감지    | ✗                                                                                 | ✓                                                                                  | ✓                                                                                   |

## 08 기능 구현 C# Windowform - 현장 카메라

### Stain을 감지하는 코드

```
Cv2.FindContours(stainMask, out var cnts, out _, RetrievalModes.External, ContourApproximationModes.ApproxSimple);
foreach (var cnt in cnts)
{
    double area = Cv2.ContourArea(cnt);
    if (area < StainMinArea || area > fabricArea * StainMaxAreaRatio) continue;
    var r = Cv2.BoundingRect(cnt);
    boxes.Add(new DetectBox
    {
        ClassName = "stain",
        Confidence = 1.0f,
        X = ox + fb.X + r.X,
        Y = oy + fb.Y + r.Y,
        W = r.Width,
        H = r.Height
    });
}
```

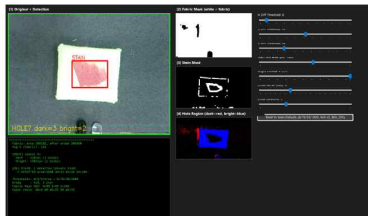
### 옷감 부분을 HSV 색 공간으로 전환 코드

```
Cv2.CvtColor(fabricRoi, fabricHsv, ColorConversionCodes.BGR2HSV);
```

### Stain을 검출하기 위해 현재 픽셀이 주변 평균 색과 차이를 계산하는 코드

```
Cv2.Absdiff(ch1[0], ch2[0], diffH);
Cv2.Absdiff(ch1[1], ch2[1], diffS);
Cv2.Absdiff(ch1[2], ch2[2], diffV);
```

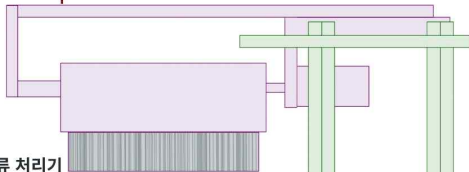
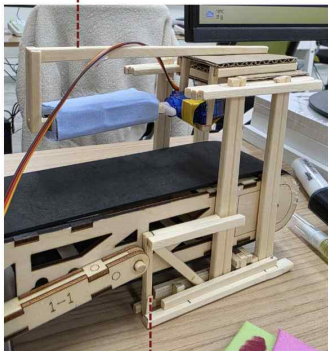
### 얼룩 감지를 위해 임계값 설정 하는 화면



### 설정 순서

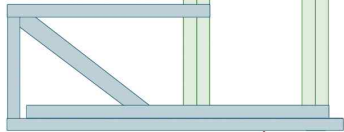
1. 임계값을 설정
2. 픽셀 값을 계산 및 설정
3. 가우시안 블러 처리

## 08 기능 구현 하드웨어 구성\_분류기



### 분류 처리기

서보모터를 이용하여 신호를 받아 불량품을  
쳐낼 수 있도록 마찰력이 강하며 컨베이어  
벨트를 손상시키지 않는 솔로 제작

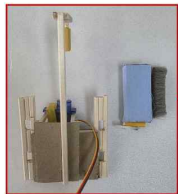


### 지지대

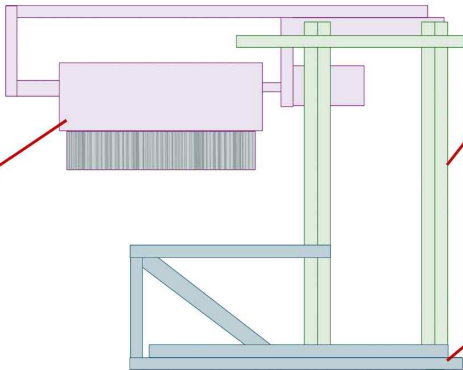
컨베이어 벨트 끝에서 고정 시킬수 있도록 제작



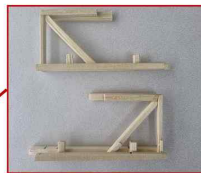
## 08 기능 구현 하드웨어 구성-분류기



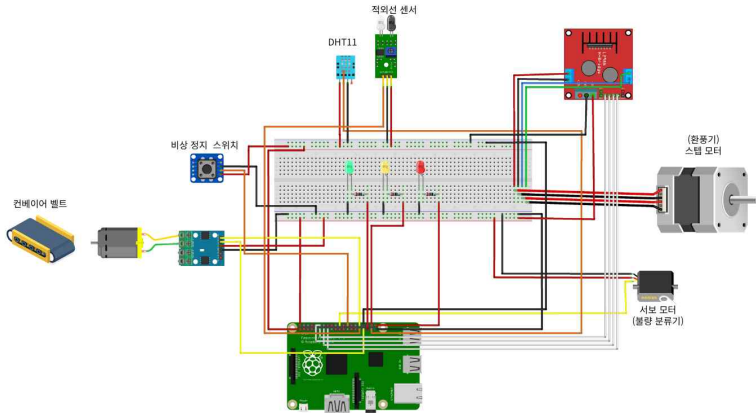
분류 처리기



지지대



## 08 기능 구현 하드웨어 구성\_라즈베리 파이\_회로도



## 08 기능 구현

### 하드웨어 구성\_라즈베리 파이와 MFC의 통신 프로토콜, 컨베이어 벨트

#### 예시

|   |      |     |      |               |
|---|------|-----|------|---------------|
| 1 | STX  | 2   | 0x02 | 패킷 시작 표시      |
| 2 | CMD  | 16  | 0x10 | 상태 보고 명령      |
| 3 | LEN  | 5   | 0x05 | 데이터 길이        |
| 4 | TEMP | 28  | 0x1C | 온도 정수값 28°C   |
| 5 | HUMI | 45  | 0x2D | 습도 정수값 45%    |
| 6 | ULTR | 82  | 0x52 | 온도/습도 소수점 데이터 |
| 7 | MOTR | 1   | 0x01 | 모터 속도 1단      |
| 8 | CS   | 119 | 0x77 | 체크섬           |
| 9 | ETX  | 3   | 0x03 | 패킷 종료 표시      |

온도 28.5°C, 습도 45.2%, 모터 속도 1단 상태를 9바이트 패킷으로  
구성하여 RS232 통신으로 전송

#### 최종 패킷

02

10

05

1C

2D

52

01

77

03

- 패킷 기반 통신 구조
- XOR 체크섬 적용
- PWM 기반 속도 제어
- 실시간 센서 데이터 전송

$CS = CMD \wedge LEN \wedge TEMP \wedge HUMI \wedge ULTR \wedge MOTR$

#### 컨베이어벨트 속도 제어

```
def set_speed(self, speed):  
    if speed == MOTOR_EMG or speed == MOTOR_OFF:  
        self._speed = MOTOR_OFF  
        self._duty = 0  
        self._stop_pwm()  
  
        lgpio.gpio_write(self._h, self._dir_pin, 0)  
    else:  
        self._speed = speed  
        self._duty = DC_SPEED_DUTY.get(speed, 50)  
  
        lgpio.tx_pwm(self._h, self._pwm_pin, DC_PWM_FREQ, self._duty)  
        self._pwm_on = True
```

## 08 기능 구현 하드웨어 구성\_라즈베리 파이를 사용해, IR센서기반 컨베이어 벨트 제어

### IR센서기반 컨베이어 벨트 동작 제어

```
def _run(self):
    while not self._stop_event.is_set():
        if lgpio.gpio_read(self._h, self._pin) == 0:

            self._buzzer.beep(0.1, 1)

            time.sleep(0.05)

            current_speed = self._dc.get_speed()
            if current_speed != MOTOR_OFF:
                self._dc.set_speed(MOTOR_OFF)
                set_leds(self._h, 0, 1, 0)

                pkt = build_packet(CMD_STATUS_REPORT, 0, 0, 0xFF, MOTOR_OFF)
                self._ser.write(pkt)

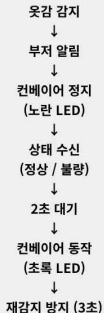
                time.sleep(2.0)

                self._dc.set_speed(current_speed)
                self._last_resume_time = time.time()
                if self._on_resume is not None:
                    self._on_resume(self._last_resume_time)
                set_leds(self._h, 1, 0, 0)

                time.sleep(3.0)

            time.sleep(0.1)
```

#### 옷감 감지 기반 컨베이어, IR 제어 흐름



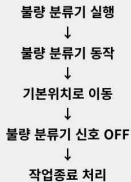
## 08 기능 구현 하드웨어 구성\_라즈베리 파이를 사용한 불량 분류

### 불량 분류 제어

```
def reject(self):  
    with self._lock:  
        if self._busy:  
  
            return  
        self._busy = True  
    threading.Thread(target=self._do_reject, daemon=True).start()
```

```
def _do_reject(self):  
    try:  
        self._set_angle(SERVO_DUTY_REJECT)  
  
        time.sleep(SERVO_HOLD_SEC)  
  
        self._set_angle(SERVO_DUTY_HOME)  
  
        time.sleep(SERVO_RETURN_SEC)  
  
        lgpio.tx_pwm(self._h, self._pin, SERVO_PWM_FREQ, 0)  
  
    finally:  
        with self._lock:  
            self._busy = False
```

#### 불량 감지 기반 불량 분류기 제어 흐름



## 08 기능 구현 하드웨어 구성\_라즈베리 파이를 사용해 온습도 센서를 이용하여 환풍기 구현

### 환풍기 제어

```
if now - last_report >= 2.0:
    temp, humi = dht.read()

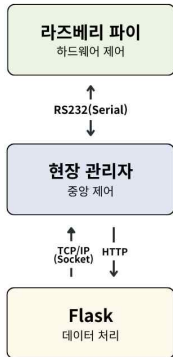
    if temp >= 60.0:
        if not stepper.is_on():
            print(f"[AUTO] ondo warring ({temp:.1f}C)! fan on")
            stepper.set_on(True)
    else:
        if not fan_manual_on and stepper.is_on():
            print(f"[AUTO] ondo Recovery ({temp:.1f}C). fan stop")
            stepper.set_on(False)
```

#### 환풍기 제어 흐름

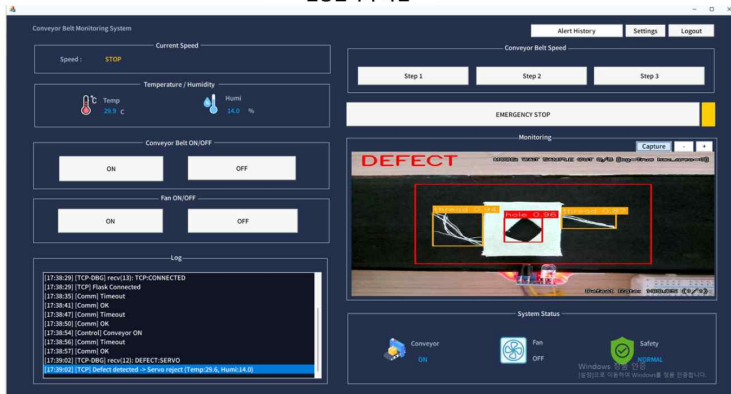
2초 간격으로 온도 확인  
↓  
60도 이상이면 환풍기 작동  
↓  
60도 이하로 내려가면  
환풍기 작동 정지

## 08 기능 구현 MFC\_통신 구조 및 제어 화면

### 통신 구조



### 현장관리자 화면



## 08 기능 구현 MFC\_비상정지

### 시스템 전체 흐름도



### 비상정지 패킷 전송

```
bool CSerialComm::SendEmergencyStop()  
{  
    BYTE pkt[PACKET_SIZE];  
    BuildPacket(pkt, CMD_EMG_STOP, MOTOR_EMG_STOP);  
    return SendPacket(pkt, PACKET_SIZE);  
}
```



### 비상정지 패킷 실행처리



```
id CConveyorSystemDlg::OnBtnEmergencyStop()  
{  
    m_serial.SendEmergencyStop();  
    UpdateSpeedLabel(); UpdateStatusLabel();  
    if (CWnd* p = GetDlgItem(IDC_STATIC_EMERGENCY_COLOR))  
        p->Invalidate();  
    SetTimer(TIMER_EMG_AUTO_RELEASE, (UINT)(m_appSettings.nEmgCooldownSec + 1000), nullptr);  
    m_dbManager.InsertAlertAsync(0, 0, "STOP", "Emergency stop button pressed");  
}
```

### RS232 시리얼 패킷 구조

| STX    | CMD    | DATA   | ETX    |
|--------|--------|--------|--------|
| 시작 바이트 | 명령     | 데이터    | 종료 바이트 |
| 1 Byte | 1 Byte | N Byte | 1 Byte |

### 현장관리자 → Flask HTTP POST (비동기 워커 스레드)

```
void CHttpClient::PostStatusAsync(float temp, float humi, int distance,  
    int motorStep, bool conveyorOn, bool fanOn, bool emgStop)  
{  
    std::ostringstream oss;  
    EnterCriticalSection(&m_csQueue);  
    while (!m_postQueue.empty()) m_postQueue.pop();  
    m_postQueue.push(oss.str());  
    LeaveCriticalSection(&m_csQueue);  
    SetEvent(m_hQueueEvent);  
}
```



## 08 기능 구현 MFC\_불량 검출

### 불량 검출 흐름도



### 불량 판정 결과 수신



### Flask → 현장관리자 TCP/IP 수신 처리

```
LRESULT CConveyorSystemDlg::OnTcpReceived(WPARAM, LPARAM)
{
    else if (data.find("DEFECT:SERVO") == 0)
    {
        m_serial.SendDefectServo();
        m_dbManager.InsertAlertAsync(m_fTemp, m_fHumi, "DEFECT",
            "Defect detected - servo reject activated");
    }
}
```

### 현장 관리자 → 라즈베리 파이 제어 흐름

- RS232(Serial) 통신 사용
- 불량 발생 시 제어 명령 전송
- 라즈베리 파이에서 서보/컨베이어 동작 수행

[12:08:42] [TCP-DBG] recv(12): DEFECT:SERVO

[12:08:42] [TCP] Defect detected -> Servo reject (Temp:27.6, Humi:20.0)

## 08 기능 구현 MFC\_로그인 및 설정 다이얼로그

로그인



```
bool CLoginDlg::ValidateLogin(const CString& id, const CString& pw)
{
    for (const auto& acc : m_accounts)
    {
        if (acc.Id.CompareNoCase(id) == 0 && acc.Pw == pw)
            return true;
    }
    return false;
}
```

```
void CSettingsDlg::OnOK()
{
    m_pSettings->fTempWarning = m_sliderTempWarn.GetPos() / 10.0f;
    m_pSettings->fTempDanger = m_sliderTempDanger.GetPos() / 10.0f;

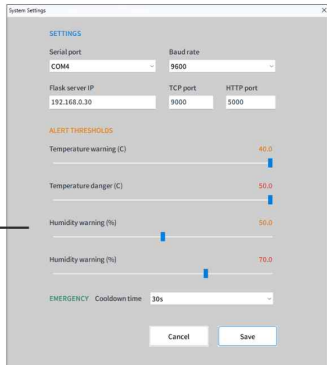
    if (m_pSettings->fTempWarning >= m_pSettings->fTempDanger)
    {
        MessageBox(_T("Temperature warning must be lower than danger threshold."),
            _T("Validation Error"), MB_OK | MB_ICONWARNING);
        return;
    }

    m_pSettings->SaveToFile(_T(".\\myconveyor_settings.ini"));

    CDialogEx::OnOK();
}
```

설정값 검증 및 저장 처리

설정 다이얼로그



## 08 기능 구현 MFC\_DB 및 PDF 리포트

- DB 저장 시점
- WARNING/DANGER 발생 시
  - EMERGENCY STOP 시

Alert History

Event Log

Today's Total: 10      Warning: 6      Danger: 3      Emergency Stop: 1

| No. | Time                | Temp (C) | Humi (%) | Status  | Content                         |
|-----|---------------------|----------|----------|---------|---------------------------------|
| 1   | 2026-04-28 11:55:50 | -        | -        | STOP    | Emergency stop button pressed   |
| 2   | 2026-04-28 11:55:30 | 28.5     | 42.0     | WARNING | Humi 42.0% warning (limit: 40%) |

`m_dbManager.InsertAlert(m_fTemp, m_fHumi, "DANGER", buf);`

Refresh      Reset

### DB 기반 PDF 리포트 생성

```
CString CReportGenerator::GenerateDailyReport(const CString& dateStr)
{
    AlertSummary summary = m_pDB->GetTodaySummary();
    TempHumiStats stats = CalcStats(m_pDB, ds);
    std::vector<AlertRecord> alerts = FilterAlertsByDate(m_pDB, ds, 30);

    std::string page = BuildPage(ds, op, summary, stats, alerts);

    return fn;
}
```

### CONVEYOR BELT DAILY OPERATION REPORT

#### 1. ALERT SUMMARY

| Category       | Count |
|----------------|-------|
| Total Alerts   | 10    |
| WARNING        | 6     |
| DANGER         | 3     |
| EMERGENCY STOP | 1     |

#### 2. TEMPERATURE / HUMIDITY STATISTICS

| Metric      | Min    | Max    | Average |
|-------------|--------|--------|---------|
| Temperature | 28.3 C | 28.9 C | 28.7 C  |
| Humidity    | 42.0 % | 71.0 % | 52.1 %  |

#### 3. RECENT ALERT HISTORY

| No. | Time     | Status  | Temp | Humi | Description                     |
|-----|----------|---------|------|------|---------------------------------|
| 1   | 11:55:50 | STOP    | 0.0  | 0.0  | Emergency stop button pressed   |
| 2   | 11:55:30 | WARNING | 28.5 | 42.0 | Humi 42.0% warning (limit: 40%) |
| 3   | 11:55:22 | WARNING | 28.5 | 42.0 | Humi 42.0% warning (limit: 40%) |
| 4   | 11:55:00 | WARNING | 28.3 | 42.0 | Humi 42.0% warning (limit: 40%) |
| 5   | 11:55:00 | WARNING | 28.7 | 47.0 | Humi 47.0% warning (limit: 40%) |
| 6   | 11:55:06 | DANGER  | 28.8 | 68.0 | Humi 68.0% exceeded (limit: 50) |
| 7   | 11:55:14 | DANGER  | 28.9 | 71.0 | Humi 71.0% exceeded (limit: 50) |
| 8   | 11:55:12 | DANGER  | 28.7 | 70.0 | Humi 70.0% exceeded (limit: 50) |
| 9   | 11:55:11 | WARNING | 28.8 | 43.0 | Humi 43.0% warning (limit: 40%) |
| 10  | 11:55:05 | WARNING | 28.8 | 43.0 | Humi 43.0% warning (limit: 40%) |

## 08 기능 구현 Flask

### 1. 로그인

#### 회원 인증 및 세션 관리

실시간 문명 통

컨베이어 라인  
중앙 관제 시스템

MFC 에지 컨트롤러 연동, 불량 검지,  
센서 모니터링 및 비상정지 제어를  
통한 관리합니다.

현재 접속자

1명

공통 생산량

1,284 개

불량률

2.3%

현재 온도

24.1 °C

로그인

계정 정보를 입력하여 시스템에 접속하세요

이미 다른 기기에서 로그인 중입니다.  
기존 세션을 종료하고 여기서 로그인하시겠습니까?

취소

강제 로그인

아이디

admin

비밀번호

\*\*\*

☐ 로그인 상태 유지

로그인

127.0.0.1:5000의 메시지

다른 기기에서 로그인되어 현재 세션이 종료됩니다.

확인

SSL 암호화 통신

로그인 인증 적용

DB에서 사용자 정보를 조회하여  
존재하지 않는 계정은 로그인 차단

```
member = Member.query.get(user_id)
if not member or member.password != password:
    return jsonify({'status': 'error', 'message': '아이디 또는 비밀번호 불일치'}), 401
```

1차) 중복 로그인 감지 → 유저에게 선택권 제공

```
if is_user_online(member.id):
    if not force:
        print(f"[AUTH] 중복 로그인 감지: {member.id} (유저 선택 대기)")
        return jsonify({
            'status': 'duplicate_login',
            'message': '이미 다른 기기에서 로그인 중입니다.'
        }), 409
```

2차) 사용자 선택 (유지 / 강제 로그인)

```
else:
    print(f"[AUTH] 강제 로그인 요청: {member.id} → 기존 세션 종료")
    _force_logout_via_socket(member.id)
    _checkout(member.id)
    unregister_session(member.id)
```

# 08 기능 구현 Flask

## 2. 대시보드 (센서 데이터)

모니터링 대시보드

센서 데이터 및 시스템 상태 실시간 확인



Socket.IO 실시간 데이터 전송

센서 → Flask → Socket.IO → 웹 대시보드

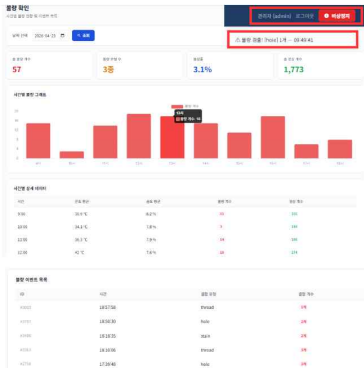
```
@api_bp.route('/status', methods=['POST'])
def api_post_status():

    if _socketio:
        _socketio.emit('sensor_update', {
            'temp': _mfc_status['temp'],
            'humidity': _mfc_status['humi'],
            'timestamp': _mfc_status['updated_at']
        },
        )

        _socketio.emit('conveyor_status', {
            'status': 'on' if conveyor_on else 'off',
            'speed': _mfc_status['speed'],
            'fan': _mfc_status['fan'],
            'emg_stop': _mfc_status['emg_stop'],
            'led_state': led_state,
            'timestamp': _mfc_status['updated_at']
        },
        )
```

## 08 기능 구현 Flask

### 3. 불량 결과 수신 및 전송



현장 카메라에서 검사 결과(정상/불량) 전송 → Flask에서 수신  
→ DB 저장 → 통계 및 명령 전달

```
@api_bp.route('/defect', methods=['POST'])  
def new_defect():
```

↓ 데이터 저장

```
try:  
    db.session.add(event)  # 정상/불량 여부를 포함한 모든 검사 결과를 DB에 저장  
    db.session.commit()
```

↓ 명령 전달

```
if defect_type != 'normal':  # 핵심 조건  
    try:  
        from tcp_server import send_defect_servo  
        send_defect_servo()  # 실제 실행  
    except Exception:
```

## 08 기능 구현 Flask

### 4. 영상 프레임 및 판별 결과 수신

현장 카메라에서 프레임 단위 영상과 판별 결과를 함께  
수신하여 웹으로 실시간 전달

```
@app.route('/predict', methods=['POST'])
def predict():
```



```
img_base64 = data['image_data']
```



```
socketio.emit('video_frame', {'frame': img_base64})
```

현장 카메라 (영상 분석) → Flask /predict → Socket.IO → 웹 화면 출력

### 5. TCP / IP로 현장 관리자에게 명령 전송

불량 발생 시 실제 장비를 제어하는 명령 전송부

```
def broadcast_to_mfc(message):
```

현장 관리자에게 명령

⋮

```
for client_sock in _mfc_clients:
```

```
try:
```

```
client_sock.sendall(message.encode('utf-8'))
```

← 실제 전송

```
print(f"[TCP] ✓ MFC에 전송 성공: {message.strip()}")
```

```
sent += 1
```

```
except Exception as e:
```

```
print(f"[TCP] X MFC 전송 실패: {e}")
```

```
dead.append(client_sock)
```



TCP / IP 전송

```
def send_defect_servo():
```

```
    broadcast_to_mfc("DEFECT:SERVO\n")
```

불량 발생 시 전달되는 제어 명령

검출 → 판단 → 물리적 제어까지 이어지는 자동화 시스템 구현

## 08 기능 구현 Flask

### 6. 모니터링 / 웹 영상 출력

모니터링

실시간 영상 스트리밍



Socket.IO로 영상 프레임 실시간 수신

현장 카메라에서 전달된 영상을 Socket.IO로 수신하여 웹에 표시

```
socket.on('video_frame', function(data) {  
    const frame = data.frame;  
    if (!frame) return;  
  
    const img = document.getElementById('liveFrame');  
    const noSignal = document.getElementById('noSignal');  
  
    // 첫 프레임 수신 시  
    if (!isStreaming) {  
        isStreaming = true;  
        img.style.display = 'block';  
        noSignal.style.display = 'none';  
        document.getElementById('live-dot').classList.add('live');  
        setConnected();  
    }  
  
    img.src = 'data:image/jpeg;base64,' + frame;
```

Flask는 영상 분석을 수행하지 않고, 전달된 영상을 화면에 출력하는 역할만 수행



## 08 기능 구현 Flask

### 7. 회원 관리

관리자 권한시 회원 관리

HTTP 요청(fetch) → Flask → DB 처리 → JSON 응답 → 목록 갱신

회원 관리

회원 추가

아이디 \*

비밀번호 \*

이름

연도-월-일

📅

직급

+ 추가

회원 목록

5명

| 회원      | 아이디   | 생년월일       | 직급      | 권한  | 가입일        | 관리    |
|---------|-------|------------|---------|-----|------------|-------|
| 관리자     | admin | 1995-03-15 | 관리자     | 관리자 | 2026-04-24 | 수정 삭제 |
| 작업자1    | user1 | 2026-04-22 | 라인 A 담당 | 일반  | 2026-04-24 | 수정 삭제 |
| 작업자2    | user2 | 2026-04-22 | 라인 B 담당 | 일반  | 2026-04-24 | 수정 삭제 |
| 현장 관리자1 | user3 | 2026-04-22 | 설비 총괄   | 일반  | 2026-04-24 | 수정 삭제 |

작업자1 (user1) 로그인



● 접근 권한 없음

회원 관리는 관리자만  
이용할 수 있습니다

이 페이지는 관리자 권한이 필요합니다.  
접근이 필요하다면 관리자에게 문의하세요.

← 대시보드로 돌아가기

#### 1) 회원 조회

```
function loadMembers() {  
  fetch('/api/members')
```

DB → Flask → 웹

#### 2) 회원 추가

```
function addMember() {  
  fetch('/api/member/add', {  
    method: 'POST', headers:
```

사용자 입력 → Flask → DB 저장

#### 3) 회원 수정

```
fetch('/api/member/update', {  
  method: 'POST',  
})
```

#### 4) 회원 삭제

```
function deleteMember(id) {  
  fetch('/api/member/delete', {  
    method: 'POST',
```

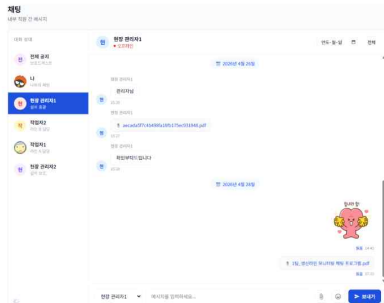
#### 5) 회원 목록 갱신

```
loadMembers();  
[ 'm-id', 'm-pw', 'm-name', 'm-birth', 'm-position' ].forEach(id  
  showToast('success', res.message || '회원이 추가되었습니다.');
```

## 08 기능 구현 Flask

### 8. 실시간 채팅

#### 작업자간 실시간 소통 기능



작업자 → Flask → DB 저장 → Socket.IO → 상대 사용자  
→ 실시간 메시지 전달 및 데이터 저장

#### 1) 채팅 전송

```
function sendMsg() {  
  const msg = input.value.trim();  
  
  socket.emit('chat_message', {  
    receiver_id: receiverId,  
    message: msg  
  });  
}
```

작업자 → Flask → DB 저장 → Socket.IO  
→ 상대 사용자

#### 2) 채팅 수신

```
socket.on('receive_message', function(data) {  
  const selectedId = document.getElementById('recipient').value;
```

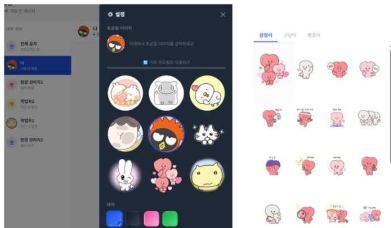
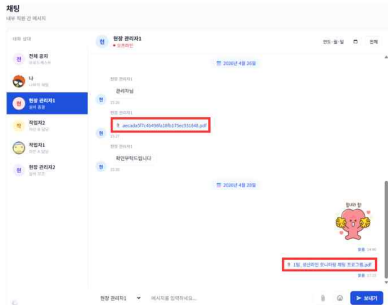
Flask → 작업자 (메시지 받는 코드)

#### 3) 채팅 화면 출력

```
container.appendChild(el);  
function addMessageToScreen(data) {
```

## 08 기능 구현 Flask

### 8. 실시간 채팅



사용자 프로필 이미지 및 이모티콘 기반 채팅 UI 구성

### 파일 전송

```
const formData = new FormData();
formData.append('file', file);
formData.append('receiver_id', receiverId);

fetch('/api/chat/upload', {
  method: 'POST',
  body: formData
})
```

파일 업로드 요청(HTTP) → Flask → 서버 저장 후 상대 사용자에게 전달

## 09 시연 영상

---



CONVEYOR SYSTEM

ID

Pin

Login

Windows 침입 차단  
(탐침으로 이동하여 Windows를 등록하십시오.)

스마트 공장

### 컨베이어 라인 중심 관리 시스템

라인 온도 관리, 속도 조절, 장애 진단 등  
라인 운영을 위한 핵심 관리 시스템

| 구분    | 현재 온도  | 설정 온도  |
|-------|--------|--------|
| 라인 온도 | 24.1°C | 25.0°C |
| 라인 속도 | 1.2m/s | 1.5m/s |

2024.04.26 17:28:57

#### 로그인

라인 관리자 권한을 입력하여 시스템에 접속하십시오

#### 아이디

아이디를 입력하십시오

#### 비밀번호

비밀번호를 입력하십시오

☐ 비밀번호를 잊으셨습니까?

로그인

© 2024 스마트 공장 | 모든 권리 보유

스마트 컨베이어 중심 스마트 공장 | 스마트 팩토리 솔루션 | 스마트 물류 시스템

## FABRIC DEFECT DETECTOR

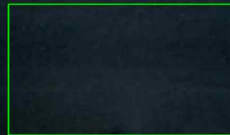
모니터링 (컨베이어)

검출 상태: WAIT

검출 로그

CONNECTED

NO FABRIC 1912 (log=Fabric Defect=0)



| 시간 | 종류 | 값 |
|----|----|---|
|    |    |   |



## 10 문제점 및 개선 사항



### 다양한 기술 스택

#### 문제점

- Flask, MFC, C# WinForm, 라즈베리파이 등 다양한 언어 및 프레임워크 동시 사용
- 기술 숙달 및 연동에 예상보다 많은 시간 소요

#### 개선사항

- 향후 사전 기술 조사 및 준비 기간 확보를 통한 지연 최소화 계획



### YOLO 데이터 부족

#### 문제점

- 3종류 불량 검출 목표 대비 데이터셋 부족
- YOLO 검출 정확도 미달

#### 개선사항

- YOLO 학습 대상 2종류로 축소, 나머지 1종류는 OpenCV 색상 이상 검출로 대체
- 밝기·대비 조절 등 데이터 증강을 통한 학습 데이터 확보 및 성능 개선



### 일정 관리

#### 문제점

- 기술 숙달 지연과 데이터셋 부족 문제의 동시 발생

#### 개선사항

- 팀원들의 적극적인 대안 제시 및 성실한 참여로 일정 내 완료

# 11 느낀점

이준석

다양한 기술 스택을 하나의 시스템으로 통합하는 과정에서 설계의 중요성을 체감하였고, 사전 구조 설계가 개발 효율에 큰 영향을 준다는 것을 배웠습니다.

김수영

Serial, TCP, HTTP를 역할에 맞게 분리 설계하는 것의 중요성을 느꼈고, MFC에서 UI와 통신을 연결하면서 이벤트 처리와 상태 관리가 실제 시스템에서 핵심적인 요소임을 체감했습니다.

김동호

불량 검출을 위해 불량 라벨링을 맡았던 것이 기억에 남습니다. 수백장이 되는 데이터를 라벨링 하는 작업과 YOLO 학습이 쉽지 않았습니다. 또한 학습 시키는 과정에서 GPU와 CPU의 학습 속도 차이를 체감 했으며 장비의 중요성을 깨달았습니다.

이승환

소프트웨어와 하드웨어를 통합하여 하나의 시스템으로 구성하는 과정에서 각 요소 간의 연동과 타이밍 제어의 중요성을 이해할 수 있었습니다. 이번 프로젝트를 통해 단순한 제어를 넘어서 통신, 하드웨어, 소프트웨어가 결합된 시스템을 구현하는 경험을 할 수 있었으며, 실제 환경에서 발생하는 다양한 문제를 해결하는 능력을 기를 수 있었습니다.

장효준

C# Winform으로 Flask와 연결을 하면서 데이터 한 번 보낸다는 게 이렇게 힘들 줄 몰랐습니다. 코드를 수정해도 원하는 대로 이루어지지 않아서 코드를 구현하는 과정에서 어려움을 겪었고 다른 팀원들과 협업을 하여 문제를 해결했기에 다행이라는 생각이 들었습니다.

최지윤

Flask를 처음 사용하면서 구조 이해와 통신 과정이 어려웠지만, 오류를 해결하는 과정에서 전체 흐름을 이해할 수 있었고 통신 구조의 중요성도 느낄 수 있었습니다.

## 12 질문과 답변

---



**프로젝트에 대해 궁금하신 점이 있으신가요?**  
자유롭게 질문해 주세요.



**경청해주셔서  
감사합니다.**

---